

Student Management System

script.js – Detailed Explanation & Viva Questions

This document provides a detailed explanation of the script.js file used in the Student Management System. It explains how the code works internally, how different parts communicate with each other, and includes viva/interview questions with answers.

1. Overview of script.js

The script.js file acts as the controller of the frontend. It connects the HTML user interface with the backend REST API, handles user actions, performs CRUD operations, and updates the UI dynamically without reloading the page.

2. API URL and DOM Element Selection

The API_URL stores the backend endpoint. DOM elements are selected so JavaScript can manipulate the form, table, buttons, and modal dynamically.

```
const API_URL = 'http://localhost:3000/api/students';

const form = document.getElementById('studentForm');
const table = document.getElementById('studentsTable');
const cancelBtn = document.getElementById('cancelBtn');
```

3. Fetching and Displaying Students (READ)

This function sends a GET request to the backend to retrieve all students. The table is cleared before inserting new rows to avoid duplication.

```
async function fetchStudents() {
  const res = await fetch(API_URL);
  const data = await res.json();

  table.innerHTML = '';

  data.data.forEach(student => {
    table.innerHTML += `
    <tr>
      <td>${student.id}</td>
      <td>${student.full_name}</td>
      <td>${student.age}</td>
      <td>${student.level}</td>
    </tr>`;
  });
}
```

4. Form Submission – Create & Update

The form submission logic handles both creating and updating students. If an ID exists, the student is updated; otherwise, a new student is created.

```
form.addEventListener('submit', async e => {
  e.preventDefault();

  const id = studentId.value;

  const student = {
    full_name: full_name.value,
    age: age.value,
    address: address.value,
```

```

        level: level.value,
        phone_number: phone_number.value,
        joined_date: joined_date.value,
        expected_graduation_date: expected_graduation_date.value
    };

    if (id) {
        await fetch(`${API_URL}/${id}`, {
            method: 'PUT',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify(student)
        });
    } else {
        await fetch(API_URL, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify(student)
        });
    }

    form.reset();
    fetchStudents();
});

```

5. Viewing Student Details (Modal Popup)

The modal popup improves user experience by displaying student details without navigating away from the page.

```

async function viewStudent(id) {
    const res = await fetch(`${API_URL}/${id}`);
    const student = (await res.json()).data;

    modalBody.innerHTML = `
        <p>Name: ${student.full_name}</p>
        <p>Age: ${student.age}</p>
        <p>Level: ${student.level}</p>
    `;

    studentModal.style.display = 'flex';
}

```

6. Edit and Delete Operations

Edit mode pre-fills the form with existing data, while delete removes a student after confirmation.

```

async function editStudent(id) {
    const res = await fetch(`${API_URL}/${id}`);
    const student = (await res.json()).data;

    studentId.value = student.id;
    full_name.value = student.full_name;
}

async function deleteStudent(id) {
    if(confirm('Are you sure?')) {
        await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
        fetchStudents();
    }
}

```

7. Viva / Interview Questions & Answers

Q: What is the purpose of script.js?

A: It controls frontend logic, handles CRUD operations, communicates with the backend API, and dynamically updates the UI.

Q: Why is async/await used?

A: Async/await is used to handle asynchronous operations like API calls in a readable way.

Q: What is CRUD?

A: CRUD stands for Create, Read, Update, and Delete.

Q: Why is fetchStudents() called after every operation?

A: To keep the UI synchronized with the backend data.

Q: What is the role of the modal popup?

A: It displays student details without navigating away from the page, improving UX.

Q: How does the code know when to update or create a student?

A: It checks whether the hidden studentId field has a value.

Q: Why do we use event.preventDefault()?

A: To prevent page reload and allow dynamic behavior.

Q: What happens when the page loads?

A: fetchStudents() is called to load existing student data.

Conclusion: The script.js file follows modern JavaScript practices and demonstrates a complete frontend CRUD implementation using REST APIs, making it suitable for academic projects and real-world applications.